

Agentenstack: externe Bausteine & Evaluationsdossier

Recherche-Zusammenfassung, Integrationsideen und Codex-Pruefauftrag fuer einen modularen Agenten-/Analyse-Stack.

Stand	04. September 2026
Ziel	Nicht blind ersetzen, sondern vorhandene Open-Source-Bausteine gegen die eigene Architektur evaluieren.
Fokus	Gateway, Registry, Eventing, Durable Jobs, Policy, Provenance, Graphmodell, Observability, Isolation, Social-Coordination-Analyse
Ergebnis	Priorisierte Shortlist + konkreter Evaluationsauftrag fuer Codex

Kernaussage: Der staerkste Architektur-Treffer ist IBM ContextForge. Dessen "Shared-Nothing Between Modules"-Ansatz entspricht praktisch dem Prinzip: Module kennen sich nicht direkt; der Core vermittelt ueber Registry, Catalog und policy-aware Routing. Fuer Skalierung und Betrieb stechen ausserdem NATS JetStream, Temporal und OPA heraus.

Inhalt & Nutzung

- 1. Executive Summary und Prioritaeten
- 2. Zielbild der moeglichen Architektur
- 3. Kandidaten im Detail
- 4. Integrationsreihenfolge / Roadmap
- 5. Bewertungsmatrix
- 6. Codex-Evaluationsauftrag (Copy/Paste)
- 7. Quellen und Links

So verwenden: Das Dokument ist als Arbeitsunterlage gedacht. Erst die Shortlist und Matrix lesen, danach den Codex-Auftrag am Ende in das Repository geben. Codex soll zunaechst nur analysieren und Vorschlaege/PoCs liefern - keine grossflaechige Migration ohne messbaren Vorteil.

1. Executive Summary und Prioritaeten

Die Recherche zeigt nicht "ein fertiges System, das alles ersetzt", sondern mehrere sehr reife Bausteine, die jeweils einen Teil eurer Plattform deutlich robuster machen koennen. Die sinnvollste Strategie ist daher composable: eigenen Core behalten, Schnittstellen standardisieren und gezielt Fremdkomponenten einsetzen.

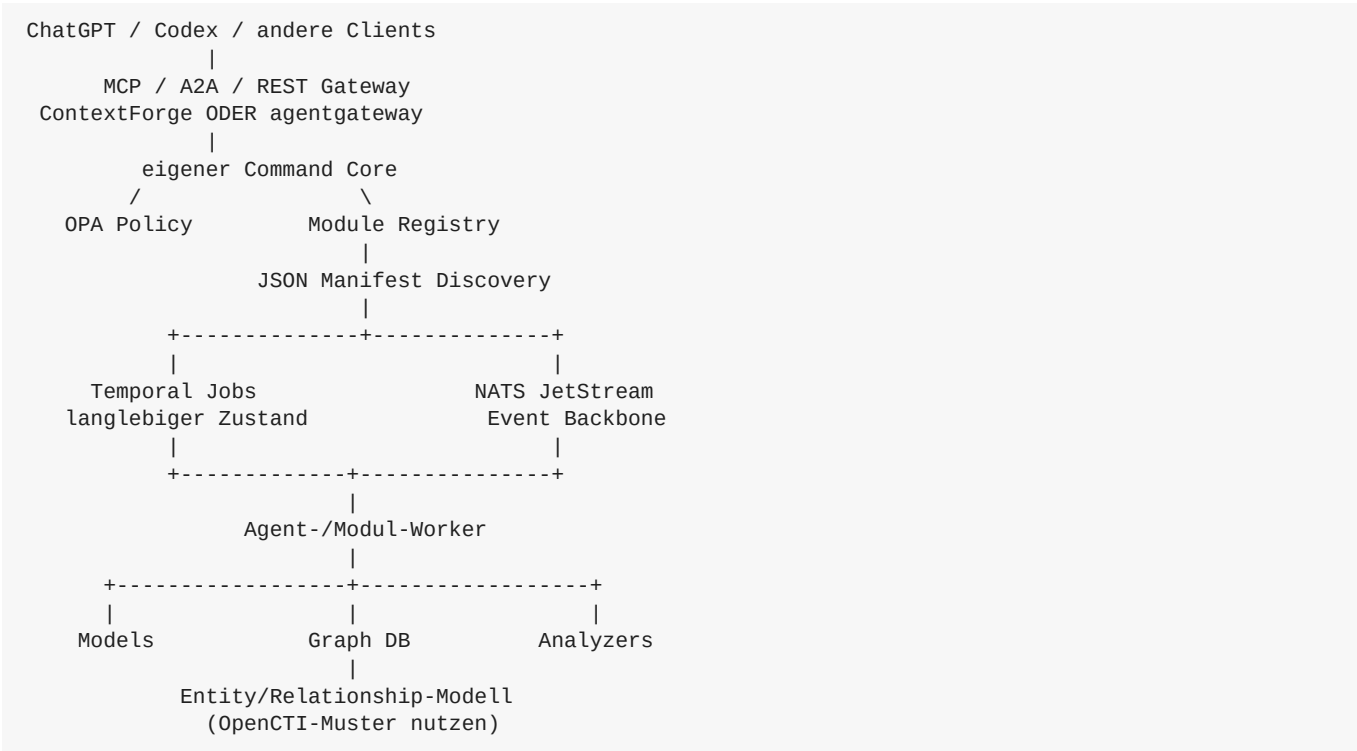
Rang	Baustein	Wofuer	Empfehlung	Warum jetzt
1	IBM ContextForge	Gateway / Registry / Plugin-Federation	Tief evaluieren	Extrem hohe Architektur- Ueberschneidung; kann viel Gateway-/Registry- Arbeit sparen.
2	NATS JetStream	Event Backbone	PoC frueh bauen	Entkoppelt viele Agenten/Module und bringt Persistence, Replay und Backpressure.
3	Temporal	Durable Jobs	Fuer lange Workflows testen	Crash Recovery, Retries, Pause/Resume und verteilte Worker statt eigener Sonderlogik.
4	Open Policy Agent (OPA)	Policy Enforcement	Sehr ernsthaft pruefen	Harte Regeln werden deterministisch erzwungen statt nur in Prompt-/Agentenlogik beschrieben.
5	OpenLineage + Sigstore Rekor	Provenance / Audit	Gezielt integrieren	Saubere Herkunftskette plus manipulationsresistenter Nachweis fuer Ergebnisse/Artefakte.
6	OpenCTI-Muster	Knowledge Graph / Entity Resolution	Design-Ideen uebernehmen	Dedup, Confidence, Beziehungen und Inference sind genau die schwierigen Graphprobleme.

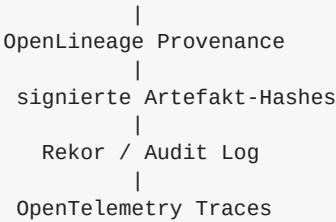
Rang	Baustein	Wofuer	Empfehlung	Warum jetzt
7	agentgateway	MCP/A2A/LLM Proxy	Gegen ContextForge benchmarken	Schlanker Linux-Foundation-naher Gateway-Kandidat fuer Routing/Governance.
8	QUT Coordination Network Toolkit	Koordinationsanalyse	Methodik/Algorithmen evaluieren	Wissenschaftlich fundierte Erkennung koordinierter Aktivitaetsmuster.
9	AgentOps / OpenTelemetry	Observability	OTel-first	Tracing/Replay/Cost-Metriken - eher als UI/Backend auf standardisierten Traces.
10	Firecracker / gVisor / Microsandbox	Isolation	Nach Workload-Klassen staffeln	Sinnvoll fuer untrusted oder agent-generierten Code; staerkere Trennung als Standardcontainer.
11	OSINTGraph	Instagram/Graph-Referenz	Code/Schema studieren	Neo4j + Social Graph + Agent Queries; eher Ideenlieferant als Kernplattform.

Entscheidungsregel: Nicht nach Feature-Liste entscheiden. Fuer jeden Kandidaten messen: 1) Welche eigene Komplexitaet faellt weg? 2) Welche neue Betriebsabhaengigkeit entsteht? 3) Wie sauber passt er an eure Manifest-/Command-Grenze? 4) Kann er in kleinen Schritten eingefuehrt und wieder entfernt werden?

2. Zielbild der moeglichen Architektur

Das folgende Zielbild ist absichtlich modular. Es ist kein Migrationsbefehl, sondern ein Referenzmodell fuer die Evaluation:





Wichtig: Die vorhandene eigene Architektur bleibt der Bezugspunkt. Externe Projekte sollen nur dort eingefuehrt werden, wo sie messbar Wartungsaufwand reduzieren, Standardschnittstellen liefern oder Zuverlaessigkeit/Skalierung deutlich erhoehen.

3. Kandidaten im Detail

3.1 IBM ContextForge

Rolle	Vorlaeufiges Urteil	Prioritaet
Gateway + Registry + Federation + Plugin Framework	Top-Kandidat: tief gegen eigenen Core mappen	SEHR HOCH

Warum relevant: ContextForge federiert MCP-, A2A-, REST- und gRPC-Dienste, besitzt Registry, Governance, Discovery, OpenTelemetry, Auth/Rate-Limits und ein Plugin-Modell. Besonders relevant ist das Architekturprinzip, dass Module nicht direkt miteinander koppeln, sondern ueber den Core vermittelt werden.

Moegliche Einbindung: Nicht sofort den eigenen Core ersetzen. Zuerst als Boundary/Gateway testen: eigene Manifest-/Command-Schicht intern behalten und ContextForge fuer externe Protokolle, Tool-Registry und Federation einsetzen. Alternativ gezielt Registry-/Plugin-Muster uebernehmen.

Trade-offs / offene Punkte: Breiter Funktionsumfang kann doppelte Abstraktionen erzeugen. Betrieb und Upgrade-Pfad muessen zur eigenen Architektur passen. Die Dokumentation zeigt zudem, dass einzelne Runtime-Experimente sich schnell aendern koennen - daher auf stabile Kernkonzepte fokussieren.

Was Codex konkret pruefen soll:

- Map: Welche eigenen Layer entsprechen ContextForge Core, Registry, Plugins, Gateway und Observability?
- Erstelle eine Overlap-Matrix: vorhanden / besser in ContextForge / eigene Speziallogik behalten.
- Pruefe, ob unsere JSON-Manifeste automatisch als ContextForge-Tools/Services registriert werden koennen.
- Baue einen minimalen Adapter-PoC mit genau einem internen Command und einem externen MCP-Aufruf.
- Messe zusaetzliche Latenz, Fehlerbilder, Debuggability und Rollback-Aufwand.
- Pruefe Lizenz, Release-Cadence, Upgrade-Kompatibilitaet und Self-Hosting-Abhaengigkeiten.

Quellen: [GitHub](#) | [Architecture](#) | [ADR-046](#)

3.2 agentgateway

Rolle	Vorlaeufiges Urteil	Prioritaet
MCP/A2A/LLM/API Gateway	Benchmark gegen ContextForge	HOCH

Warum relevant: agentgateway ist ein offener Gateway/Data-Plane fuer HTTP/gRPC und Agent-Traffic mit MCP/A2A, Auth, Governance, Routing und Observability. Die Staerke liegt eher in einem fokussierten Traffic-/Policy-Gateway als in einem grossen All-in-one-Agentenframework.

Moegliche Einbindung: Als Alternative fuer die Aussengrenze des Systems testen. Wenn die eigene Registry bereits stark ist, kann agentgateway moeglicherweise als schlankerer Proxy besser passen als ContextForge.

Trade-offs / offene Punkte: Nicht beide Gateways ohne klaren Grund stapeln. Sonst entstehen doppelte Auth-, Routing- und Telemetrieebenen.

Was Codex konkret pruefen soll:

- Identische Testfaelle gegen ContextForge ausfuehren.
- Feature-Paritaet fuer MCP/A2A, REST/gRPC, Auth, RBAC, Rate-Limits und OpenTelemetry erfassen.
- P95/P99-Latenz, Throughput, Konfigurationsaufwand und Failover vergleichen.
- Pruefen, ob der Gateway ohne Umbau des internen Command-Cores austauschbar bleibt.

Quellen: [Website](#) | [Docs](#)

3.3 NATS + JetStream

Rolle	Vorlaeufiges Urteil	Prioritaet
Event Backbone / PubSub / Durable Events	Fruerher PoC empfohlen	SEHR HOCH

Warum relevant: Bei vielen Agenten und Modulen wird direkte Punkt-zu-Punkt-Kommunikation schnell unuebersichtlich. JetStream bringt persistente Events, durable Consumers, Acknowledgements, Replay und natuerliche Backpressure. Producer und Consumer muessen nicht gleichzeitig online sein.

Moegliche Einbindung: Events wie analysis.completed, entity.resolved, artifact.created oder module.health koennen ueber Subjects publiziert werden. Der Core bleibt fuer synchrone Commands zustaendig; NATS uebernimmt asynchrone Verteilung und Fan-out.

Trade-offs / offene Punkte: At-least-once-Delivery bedeutet: Handler muessen idempotent sein. Event-Schemata, Versionierung, Dead-letter-Strategie und Retention brauchen klare Regeln.

Was Codex konkret pruefen soll:

- Entwirf ein Subject-Namensschema und Event-Envelope mit schema_version, trace_id, actor/module, timestamp und payload_ref.
- Baue einen PoC mit zwei Produzenten, drei Consumern, absichtlichem Worker-Absturz und Replay.
- Messe Backpressure und Verhalten bei Burst-Last.
- Definiere Idempotency-Key und Dedup-Strategie.
- Pruefe, welche bisherigen internen Message-/Queue-Loesungen dadurch entfallen koennen.

Quellen: [NATS](#) | [Python JetStream](#)

3.4 Temporal

Rolle	Vorlaeufiges Urteil	Prioritaet
Durable Workflow Engine	Fuer lange/mehrstufige Jobs testen	SEHR HOCH

Warum relevant: Temporal persistiert Workflow-Zustand und bietet automatische Retries, Pause/Resume, Historie und verteilte Worker. Das ist vor allem fuer Agentenlaeuft wertvoll, die lange dauern, externe Abhaengigkeiten haben oder Deployments/Crashes ueberleben muessen.

Moegliche Einbindung: Synchrone Kurzaufgaben koennen weiterhin direkt laufen. Lange Research-/Analyse-/Enrichment-Jobs werden als Temporal Workflows modelliert; externe LLM-, HTTP- und Tool-Aufrufe als Activities.

Trade-offs / offene Punkte: Temporal ist eine echte Infrastrukturkomponente mit eigenem Betriebsmodell. Deterministische Workflow-Regeln und Versionierung muessen verstanden werden. Nicht jede kleine Aufgabe braucht Temporal.

Was Codex konkret pruefen soll:

- Identifiziere bestehende Workflows, die heute eigene Retry-/Checkpoint-/Resume-Logik besitzen.
- Portiere genau einen repraesentativen Workflow als PoC.
- Simuliere Crash, Deploy, Timeout und manuelle Pause/Resume.
- Messe Code, der durch Temporal entfaellt, versus neue Infrastrukturkomplexitaet.
- Pruefe mcp-agent/Temporal nur als Referenz oder Adapter, nicht automatisch als zusaetzliches Framework.

Quellen: [Temporal](#) | [mcp-agent](#)

3.5 Open Policy Agent (OPA)

Rolle	Vorlaeufiges Urteil	Prioritaet
Policy-as-Code / deterministisches Enforcement	Sehr ernsthaft integrieren pruefen	SEHR HOCH

Warum relevant: OPA trennt Policy-Entscheidungen von Anwendungslogik. Statt dass ein Agent oder Prompt entscheidet, ob eine Aktion zulaessig ist, fragt der Core eine deklarative Policy ab und erhaelt eine reproduzierbare Entscheidung.

Moegliche Einbindung: OPA vor Tool-/Modul-/Datenzugriffe setzen. Beispiele: Welche Rolle darf welches Modul starten? Welche Capabilities darf ein Manifest anfordern? Welche Datenklasse darf an welchen Connector? Welche Ressourcenlimits gelten?

Trade-offs / offene Punkte: Rego hat Lernkurve. Policy-Input muss klein und stabil modelliert werden; OPA ersetzt kein Identitaets- oder Secrets-System, sondern entscheidet auf Basis bereitgestellter Fakten.

Was Codex konkret pruefen soll:

- Formuliere 5 reale bestehende Regeln als Rego-PoC.
- Baue einen zentralen Policy-Input-Contract, der nicht von internem Python-Objektlayout abhaengt.
- Teste fail-closed Verhalten bei OPA-Ausfall.
- Pruefe Bundle-/GitOps-Workflow und Policy-Tests im CI.
- Messe Evaluationslatenz bei hoher Tool-Call-Frequenz.

Quellen: [GitHub](#) | [Website](#)

3.6 OpenLineage + Sigstore Rekorder

Rolle	Vorlaeufiges Urteil	Prioritaet
Provenance + manipulationsresistentes Audit	Gezielte Kombination	HOCH

Warum relevant: OpenLineage liefert ein offenes Modell fuer Lauf-/Job-Lineage. Rekorder ist ein Transparency Log fuer signierte Metadaten und Hashes. Zusammen koennen sie nachvollziehbar machen, welche Inputs, Modelle, Module und Schritte ein Artefakt erzeugt haben und ob der protokollierte Nachweis nachtraeglich veraendert wurde.

Moegliche Einbindung: OpenLineage fuer interne Herkunftsdaten nutzen. Ergebnisartefakte separat speichern; Hash + Signatur + zentrale Provenance-Referenz in einen eigenen oder Rekorder-kompatiblen Transparency-Log schreiben. Keine sensiblen Rohdaten in einen oeffentlichen Log legen.

Trade-offs / offene Punkte: Rekorder stammt aus Software-Supply-Chain-Kontext und muss auf euren Use Case sauber abgebildet werden. Rekorder v1/v2-Entwicklung und Speicher-/Privacy-Modell genau pruefen.

Was Codex konkret pruefen soll:

- Definiere eine minimale Provenance-Struktur fuer jeden Agentenlauf.
- Pruefe OpenLineage-Facets fuer model_id, prompt/config hash, source refs und output hashes.
- PoC: Ein Artefakt erzeugen, Hash signieren, Log-Eintrag erstellen, spaeter Integritaet verifizieren.
- Entscheide Private Transparency Log vs. oeffentliche Instanz.
- Pruefe, welche eurer bisherigen Signatur-/Exportlogik dadurch vereinfacht wird.

Quellen: [OpenLineage](#) | [Rekorder v1](#) | [Rekorder v2](#)

3.7 OpenCTI - Architektur- und Datenmodell-Muster

Rolle	Vorlaeufiges Urteil	Prioritaet
Knowledge Graph / Entity Resolution / Confidence	Muster uebernehmen, nicht blind Plattform zweckentfremden	HOCH

Warum relevant: OpenCTI loest seit Jahren schwierige Graphprobleme: Entities und Relationships, Deduplication, Merging, Confidence, Quellen, Inference-Regeln und Daten-Segregation. Diese Muster sind fuer jede groessere OSINT-/Analyseplattform relevant.

Moegliche Einbindung: Nicht zwingend OpenCTI als Backend verwenden. Stattdessen Datenmodell, Upsert-/Dedup-Regeln, Confidence-Propagation und Inference-Patterns gegen das eigene Graphschema halten.

Trade-offs / offene Punkte: STIX ist fuer Cyber Threat Intelligence entworfen und passt semantisch nicht 1:1 auf Social-/Allgemein-OSINT. Uebernahme sollte auf Mechanismen, nicht auf unpassende Objektklassen fokussieren.

Was Codex konkret pruefen soll:

- Vergleiche eigenes Entity-/Relationship-Schema mit OpenCTI-Konzepten.
- Definiere ID Contributing Properties fuer eigene Entity-Typen.
- Pruefe Confidence- und Source-Prioritaetsregeln bei widerspruechlichen Beobachtungen.
- Baue einen Merge-/Dedup-Testkorpus mit Aliasen und konfligierenden Quellen.
- Pruefe Inference-Regeln und markiere abgeleitete Beziehungen explizit als inferred.

Quellen: [OpenCTI](#) | [Docs](#)

3.8 QUT Coordination Network Toolkit

Rolle	Vorlaeufiges Urteil	Prioritaet
Methodik fuer koordinierte Social-Media-Aktivitaet	Algorithmik/Methodik evaluieren	MITTEL-HOCH

Warum relevant: Das Toolkit modelliert co-occurring activity als Netzwerke und wurde als wissenschaftlicher Rahmen zur Erkennung und Analyse koordinierten Verhaltens entwickelt. Das ist methodisch staerker als nur einen einzelnen "Bot Score" zu betrachten.

Moegliche Einbindung: Algorithmen/Features in einen eigenstaendigen Analyse-Worker integrieren. Ergebnis als Graph/Features an den bestehenden Core zurueckgeben. Fokus auf oeffentliche, rechtmassig verarbeitete Daten und reproduzierbare Features.

Trade-offs / offene Punkte: Datenquellen und Plattform-APIs unterscheiden sich; Twitter-spezifische Annahmen muessen auf andere Plattformen validiert werden. Koordination ist nicht automatisch Inauthentizitaet.

Was Codex konkret pruefen soll:

- Extrahiere die benoetigten Eingabefelder und Graphkonstruktion.
- Pruefe, welche Features plattformneutral sind (Zeit, URL, Hashtag, Text-/Mediengleichheit, Interaktionen).
- Erstelle einen kleinen synthetischen Benchmark mit bekannten koordinierten und normalen Gruppen.
- Bewerte False-Positive-Risiko und erklaerbare Output-Metriken.

Quellen: [QUT Toolkit](#) | [GitHub](#)

3.9 OSINTGraph

Rolle	Vorlaeufiges Urteil	Prioritaet
Instagram-/Social-Graph-Referenz	Code und Datenmodell studieren	MITTEL

Warum relevant: OSINTGraph legt oeffentliche Instagram-Daten in Neo4j ab, visualisiert Profile/Beziehungen/Posts/Kommentare und erlaubt Agentenabfragen auf dem Graphen. Das ist als Referenz fuer Importpfade, Neo4j-Modellierung und Query-UX interessant.

Moegliche Einbindung: Nicht als zentralen Stack ansehen. Einzelne Adapter-, Schema- oder Query-Ideen koennen uebernommen oder isoliert getestet werden.

Trade-offs / offene Punkte: Plattformbedingungen, Zugriffsmethoden und Wartbarkeit koennen sich schnell aendern. Keine Architektur an fragile Scraper koppeln.

Was Codex konkret pruefen soll:

- Pruefe Datenmodell und Neo4j-Knoten/Kanten gegen das eigene Schema.
- Identifiziere wiederverwendbare Parser/Importer ohne Plattform-Bypass-Logik.
- Bewerte Agent-Query-Layer getrennt vom Datensammler.
- Pruefe Lizenz und Aktivitaet/Release-Qualitaet.

Quellen: [GitHub](#)

3.10 AgentOps + OpenTelemetry

Rolle	Vorlaeufiges Urteil	Prioritaet
Agent Observability	OpenTelemetry als Standard, UI austauschbar halten	MITTEL-HOCH

Warum relevant: AgentOps bietet Tracing, Session Replay, Kosten- und Performance-Metriken. Gleichzeitig ist OpenTelemetry der wichtigere Architekturanker, weil Traces/Metrics dadurch nicht an ein einzelnes Dashboard gebunden sind.

Moegliche Einbindung: Instrumentation moeglichst OTel-first gestalten. Danach AgentOps, Jaeger, Phoenix, Grafana oder andere Backends als austauschbare Auswertungsschicht testen.

Trade-offs / offene Punkte: Framework-spezifische Instrumentierung kann Lock-in erzeugen. Nicht jede Payload sollte vollstaendig geloggt werden; Datenschutz und Secrets-Redaction gehoeren in den Telemetrievertrag.

Was Codex konkret pruefen soll:

- Definiere Trace-Spans fuer command -> policy -> workflow -> tool -> artifact.
- Korrelation mit NATS event_id und Temporal workflow_id festlegen.
- Pruefe AgentOps als UI/Backend, ohne Kerncode davon abhaengig zu machen.
- Definiere Redaction fuer Secrets, personenbezogene und grosse Payloads.

Quellen: [AgentOps](#) | [OpenTelemetry](#)

3.11 Firecracker / gVisor / Microsandbox

Rolle	Vorlaeufiges Urteil	Prioritaet
Sandbox / Workload Isolation	Nach Risikoklasse staffeln	MITTEL-HOCH

Warum relevant: Wenn Agenten Code oder Drittmodule ausfuehren, ist eine staerkere Isolation als normaler Docker-Betrieb attraktiv. Firecracker bietet microVMs mit Hardware-Virtualisierung; gVisor vermittelt Syscalls ueber einen User-Space-Kernel; Microsandbox ist als self-hosted microVM-Ausfuehrung mit MCP-Bezug interessant.

Moegliche Einbindung: Nicht alles in microVMs zwingen. Trusted, interne Worker koennen leichtgewichtig bleiben; untrusted oder agent-generierter Code bekommt eine staerkere Sandbox-Klasse. Capabilities und Netzwerkfreigaben sollten pro Job deklarativ sein.

Trade-offs / offene Punkte: Firecracker braucht KVM/Linux und erhoehrt Betriebsaufwand. GPU- und Dateisystemanforderungen koennen die Wahl beeinflussen. Isolation ersetzt keine Netzwerk-/Secret-Policies.

Was Codex konkret pruefen soll:

- Klassifiziere Workloads in trusted / constrained / untrusted.
- PoC gVisor fuer schnelle Standardjobs und microVM fuer hoehere Isolation.
- Messe Startup, RAM, IO und Tool-Kompatibilitaet.
- Definiere deny-by-default Netzwerk und kurzlebige Credentials fuer Sandboxes.

- Prüfe, ob euer aktueller Kubernetes/containerd-Stack RuntimeClass sauber unterstützt.

Quellen: [Firecracker](#) | [gVisor](#) | [Microsandbox](#)

4. Empfohlene Integrationsreihenfolge / Roadmap

Phase	Ziel / Exit-Kriterium
Phase 0 - Inventar	Noch nichts integrieren. Codex erzeugt eine Ist-Architekturkarte: Layer, Prozesse, Queues, DBs, Tool-Calls, Policies, Retries, Audit, Tracing und Modul-Lifecycle.
Phase 1 - Boundary Benchmark	ContextForge vs. agentgateway gegen einen identischen internen Command testen. Ziel: entscheiden, ob ein externer Gateway echten Mehrwert bringt.
Phase 2 - Event Backbone	NATS JetStream fuer genau einen Event-Typ einfuehren. Idempotency, Replay und Backpressure testen, bevor weitere Events migriert werden.
Phase 3 - Durable Workflow	Einen langen, fehleranfaelligen Workflow in Temporal abbilden. Erst nach Crash-/Resume-Test entscheiden, ob weitere Workflows folgen.
Phase 4 - Policy	OPA mit wenigen hochwertigen Regeln vor zentrale Tool-/Modulaufrufe setzen. Policies versionieren und im CI testen.
Phase 5 - Provenance	OpenLineage-artige Run-Metadaten und signierte Output-Hashes standardisieren. Rekord/private Transparency-Log-Variante benchmarken.
Phase 6 - Graphqualitaet	OpenCTI-Muster fuer Dedup, Confidence und Inference in den eigenen Graph integrieren; kontrollierter Testkorpus.
Phase 7 - Spezialanalyse	QUT Coordination Network Toolkit und OSINTGraph als spezialisierte Analyse-/Import-Referenzen testen.
Phase 8 - Isolation	Sandbox-Klassen je nach Workload und Threat Model einfuehren. Trusted Worker nicht unnoetig verteuern.
Stop-Regel: Jede Phase bekommt ein messbares Exit-Kriterium. Wenn ein Baustein keine eigene Komplexitaet eliminiert, keinen Standardvorteil bringt oder schlechter debuggbar wird, bleibt er draussen. "Ist beliebt" ist kein Integrationsargument.	

5. Bewertungsmatrix fuer die eigentliche Repo-Analyse

Kriterium	Gewicht	Frage fuer Codex
Architektur-Fit	20%	Passt der Baustein an unsere vorhandenen Grenzen oder zwingt er uns zu einem Umbau?
Eliminierte Eigenkomplexitaet	15%	Welche eigenen Module/Retry-/Routing-/Registry-/Audit-Funktionen koennen wirklich entfallen?

Kriterium	Gewicht	Frage fuer Codex
Betriebsaufwand	10%	Welche neuen Services, Datenbanken, Cluster oder On-Call-Risiken kommen dazu?
Skalierung & Backpressure	10%	Wie verhaelt sich das System bei 10x/100x Last, Burst und Worker-Ausfaellen?
Observability & Debugging	10%	Sind Ursachen, Traces, Replays und Fehlerpfade besser sichtbar als heute?
Austauschbarkeit	10%	Kann der Baustein hinter einem Adapter verschwinden und spaeter ersetzt werden?
Daten-/Policy-Kontrolle	10%	Kann Zugriff deterministisch, nachvollziehbar und minimal privilegiert gesteuert werden?
Reife / Wartung	5%	Aktive Releases, Tests, Dokumentation, Governance, Upgrade-Pfad?
Lizenz & Self-Hosting	5%	Passt Lizenz und ist der kritische Pfad ohne proprietäre Pflichtdienste betreibbar?
Migration / Rollback	5%	Kann ein PoC parallel laufen und bei Nichtgefallen ruckstandsfrei entfernt werden?

Scoring: 0 = ungeeignet, 1 = schwach, 2 = begrenzt, 3 = brauchbar, 4 = stark, 5 = sehr stark. Codex soll neben dem Score immer konkrete Repo-Dateien/Interfaces nennen, die das Urteil belegen.

6. Codex-Evaluationsauftrag (Copy/Paste)

Hinweis: Der folgende Auftrag ist absichtlich analyse-first. Er soll verhindern, dass ein Coding-Agent aus Begeisterung sofort fuenf Frameworks in den Main-Branch zieht.

AUFGABE: Externe Architekturbausteine gegen dieses Repository evaluieren

ZIEL

Pruefe, welche der unten genannten Open-Source-Projekte unsere bestehende Architektur sinnvoll ergaenzen oder eigene Komplexitaet ersetzen koennen. Keine grossflaechige Integration und keine Migration im ersten Durchlauf. Erst Ist-Zustand verstehen, dann kleine PoCs vorschlagen.

KANDIDATEN

1. IBM ContextForge - <https://github.com/IBM/mcp-context-forge>
2. agentgateway - <https://agentgateway.dev/>
3. NATS / JetStream - <https://github.com/nats-io/nats-server>
4. Temporal - <https://temporal.io/>
5. Open Policy Agent - <https://github.com/open-policy-agent/opa>
6. OpenLineage - <https://github.com/OpenLineage/OpenLineage>
7. Sigstore Rekor / rekor-tiles - <https://github.com/sigstore/rekor> / <https://github.com/sigstore/rekor-tiles>
8. OpenCTI - <https://github.com/OpenCTI-Platform/opencti>
9. QUT Coordination Network Toolkit - <https://github.com/QUT-Digital-Observatory/coordination-network-toolkit>
10. AgentOps / OpenTelemetry - <https://github.com/AgentOps-AI/agentops> / <https://opentelemetry.io/>
11. Firecracker / gVisor / Microsandbox - <https://github.com/firecracker-microvm/firecracker> / <https://gvisor.dev/> / <https://github.com/microsandbox/microsandbox>
12. OSINTGraph - <https://github.com/XD-MHL00/OSintgraph>

SCHRITT 1 - IST-ARCHITEKTUR

Erstelle zuerst eine belastbare Karte dieses Repositories:

- Entry Points / APIs / CLI / Chat-Commands
- Core / Router / Registry / Manifest-System
- Module, Plugins, Workers und Agenten
- synchrone vs. asynchrone Kommunikation
- Queues / Events / Background Jobs
- Persistenz / Graph / Cache
- Retry, Timeout, Checkpoint, Resume
- AuthN/AuthZ/Policy
- Provenance, Signaturen, Audit, Exporte
- Logging, Metrics, Tracing
- Sandbox / Prozess- / Container-Grenzen
- Deployment und Skalierung

Nenne fuer jeden Punkt konkrete Dateien, Klassen oder Module.

SCHRITT 2 - OVERLAP

Erstelle fuer jeden Kandidaten eine Tabelle:

- Was kann das Projekt?
- Welche unserer vorhandenen Komponenten ueberlappen damit?
- Was ist bei uns bereits besser/spezifischer?
- Welche eigene Komplexitaet koennte entfallen?
- Welche neue Abhaengigkeit/Betriebslast entsteht?
- Integrationsgrenze/Adapter-Idee
- Migration in kleinen Schritten moeglich? Ja/Nein + warum
- Rollback-Strategie
- Lizenz / Self-Hosting / Release-Reife

SCHRITT 3 - SCORE

Score 0..5 und gewichtete Gesamtwertung fuer:

- Architektur-Fit 20%
- eliminierte Eigenkomplexitaet 15%
- Betriebsaufwand 10%
- Skalierung/Backpressure 10%
- Observability/Debugging 10%
- Austauschbarkeit 10%
- Daten-/Policy-Kontrolle 10%
- Reife/Wartung 5%
- Lizenz/Self-Hosting 5%
- Migration/Rollback 5%

SCHRITT 4 - KONKRETE POC-VORSCHLAEGE

Nur fuer die besten Kandidaten. Jeder PoC muss:

- klein und isoliert sein,
- keinen Main-Flow ersetzen,
- messbare Erfolgskriterien besitzen,
- einen sauberen Entfernen-/Rollback-Pfad haben.

Mindestens pruefen:

- A) ContextForge vs. agentgateway: identischer Gateway-Test.
- B) NATS JetStream: Event mit Durable Consumer, Worker-Crash, Replay und Backpressure.
- C) Temporal: ein langer Workflow mit Crash + Resume.
- D) OPA: 5 reale Regeln als Policy-as-Code, inkl. fail-closed Verhalten.
- E) Provenance: ein Output mit Run-Metadaten, Hash, Signatur und Verifikation.

SCHRITT 5 - ENTSCHEIDUNG

Jeden Kandidaten genau einer Kategorie zuordnen:

- ADOPT: stark genug fuer produktive Integration
- INTEGRATE: als klar begrenzter Baustein/Adapter nutzen
- MINE IDEAS: Architektur/Algorithmen uebernehmen, Projekt selbst nicht einbauen
- IGNORE FOR NOW: aktuell kein ausreichender Vorteil

OUTPUT

Erzeuge zunaechst nur Dokumentation/Analyse, keine grossen Codeaenderungen:

- docs/architecture/external-stack-evaluation.md
- docs/architecture/external-stack-scorecard.md

- docs/architecture/poc-plan.md

WICHTIG

- Keine Empfehlung nur aufgrund von Popularitaet oder Feature-Anzahl.
- Jede Aussage zum Ist-Zustand mit konkretem Repo-Pfad belegen.
- Bestehende spezialisierte Logik nicht ersetzen, wenn sie einen klaren projektspezifischen Vorteil hat.
- Standards und austauschbare Adapter bevorzugen.
- Wo moeglich OpenTelemetry, standardisierte Schemas und klare Core-Grenzen bevorzugen.
- Fuer Social-/OSINT-Daten nur rechtmassig und entsprechend Plattformbedingungen zugreifbare Datenpfade einplanen.

7. Quellen und Links

IBM ContextForge: <https://github.com/IBM/mcp-context-forge>

ContextForge Architecture: <https://github.com/IBM/mcp-context-forge/tree/main/docs/docs/architecture>

agentgateway: <https://agentgateway.dev/>

NATS: <https://github.com/nats-io/nats-server>

Temporal: <https://temporal.io/>

mcp-agent: <https://github.com/lastmile-ai/mcp-agent>

Open Policy Agent: <https://github.com/open-policy-agent/opa>

OpenLineage: <https://github.com/OpenLineage/OpenLineage>

Sigstore Rekor: <https://github.com/sigstore/rekor>

Sigstore Rekor v2: <https://github.com/sigstore/rekor-tiles>

OpenCTI: <https://github.com/OpenCTI-Platform/opencti>

QUT Coordination Network Toolkit: <https://github.com/QUT-Digital-Observatory/coordination-network-toolkit>

OSINTGraph: <https://github.com/XD-MHLOO/Osintgraph>

AgentOps: <https://github.com/AgentOps-AI/agentops>

OpenTelemetry: <https://opentelemetry.io/>

Firecracker: <https://github.com/firecracker-microvm/firecracker>

gVisor: <https://gvisor.dev/>

Microsandbox: <https://github.com/microsandbox/microsandbox>

Statushinweis: Dieses Dossier ist eine Evaluationsunterlage mit Stand 04.09.2026. Vor produktiver Uebernahme muessen Codex bzw. das Team aktuelle Releases, Lizenzen, Security Advisories, Breaking Changes und die tatsaechliche Repo-Kompatibilitaet erneut pruefen.